

p4SD: A lightweight port scan detection for programmable networks

Daniel Miranda
INESC TEC & University of Minho
daniel.b.miranda@inesctec.pt

Rui Monteiro
INESC TEC & University of Minho
rui.p.monteiro@inesctec.pt

João Marco C. Silva
INESC TEC & University of Minho
joaomarco@di.uminho.pt

Abstract—To address the challenge of detecting stealthy port scans in high-speed networks, this paper introduces p4SD, a lightweight anomaly detection system that identifies reconnaissance activities directly within programmable data planes. Leveraging the P4 language, p4SD uses a cyclic fingerprint buffer and frequency analysis to monitor for anomalous traffic without relying on attack signatures. The system is designed to detect both fast and slow port scans, as its method of measuring relative changes in distinct fingerprints between cycles effectively identifies both the rapid spikes from fast scans and the gradual increases from slow scans. The proof-of-concept demonstrates resource efficiency, achieving throughput close to the hardware’s theoretical limits, detecting scan activity in near real-time, and enabling timely responses to potential threats. With over 99% detection accuracy for slow scans, these findings establish p4SD as a practical and scalable solution for real-time, in-network threat detection in modern SDN environments.

I. INTRODUCTION

The continuous evolution of technology has driven the expansion of networks to meet increasing demands in multiple economic sectors. However, this growth comes with escalating cyber threats targeting information systems. Among various attack strategies, reconnaissance techniques like port scanning play a crucial role in identifying system vulnerabilities by probing transport layer ports to discover open services or applications that may be exploited.

Port states, commonly classified as *open*, *closed*, or *filtered*, serve primarily as indicators within the broader context of network reconnaissance, rather than as direct determinants of security risk. The critical concern arises not from the port state but from the presence of vulnerable or misconfigured services accessible through these ports, which can be used as pathways for cyber attacks [1].

Port scanning techniques vary in speed, evasion strategies, and exploitation of protocol weaknesses, making their detection challenging. In addition, they can also be classified according to the source and target multiplicity. *Single-source scans* originate from one IP address, while *distributed scans* use multiple sources to evade detection. Target multiplicity includes *vertical scans* (scanning multiple ports on one target), *horizontal scans* (scanning one port across multiple targets), *strobe scans* (scanning mul-

tiple ports on multiple targets), and *block scans* (scanning all ports on multiple targets) [2].

A particularly challenging evasion technique is *slow scan* due to its deliberate strategy of spreading probes over extended timeframes. By operating at a low frequency, slow scans aim to blend into normal network traffic patterns, reducing the likelihood of triggering conventional detection systems [3]. This method exploits the temporal dimension of scanning activity, making it difficult for security tools to correlate scattered probes and identify suspicious intents. As a result, slow scans demand detection techniques capable of analyzing long-term behavioral trends and correlating seemingly benign activities across extended periods.

Aiming to detect port scans effectively, anomaly detection mechanisms use statistical models or rule-based approaches to identify deviations from normal network behavior. However, due to the huge amount of data traversing current networks, these tools are predominantly stateless, meaning they analyze each network packet independently without maintaining contextual information about previous packets or flows, which limits their ability to correlate events over time and detect sophisticated or slow-moving attacks.

In Software-Defined Networking (SDN), Programmable Data Planes (PDP) enable complex traffic analysis at near line-rate performance while preserving necessary contextual information for detecting slow port scans. Within this context, the Programming Protocol-independent Packet Processors (P4) is notable for its protocol independence and fine-grained customization of packet processing, and its compatibility with protocols like OpenFlow enhances device programmability for high-performance, low-latency anomaly detection in modern networks.

In this way, this work introduces p4SD, an anomaly detection system designed specifically for port scanning detection within SDN environments. By leveraging P4’s capabilities, p4SD achieves high throughput performance close to hardware limits while employing efficient memory usage for real-time detection. This approach demonstrates the feasibility of deploying scalable cybersecurity solutions directly within programmable data planes.

The rest of this document is organized as follows. Section II presents previous works leveraging PDP to-

ward intrusion detection. Section III describes the p4SD architecture and implementation approaches. Section IV presents the environment deployed to test p4SD performance. Section V discusses the obtained results. Finally, Section VI concludes the paper with the final remarks.

II. RELATED WORK

Recent advances in PDP have enabled the creation of several P4-based Network Intrusion Detection Systems (NIDS). Early efforts, such as P4ID [4] and P4-ONIDS [5], mapped attack signatures onto P4 tables and offloaded analysis to external processors, creating potential bottlenecks and increasing system complexity in high-speed environments. More advanced solutions like EUCLID [6] perform detection entirely in the data plane, but their reliance on read-write intensive techniques (e.g., Count-Min Sketches [7], EWMA [8]) can impact performance under high loads.

Typical port scan detection leverages anomaly detection over time-series data to spot unusual network behaviors, often falling back on memory-efficient probabilistic data structures such as Bloom [9] or Cuckoo [10] filters. However, these structures face challenges in P4-based deployments due to limited support for iterative operations, and can suffer from high false positive rates.

Non-P4-based IDS, whether signature-based, anomaly-driven, or using Machine Learning (ML) [11], [12], almost always depend on external processing components. This dependence limits real-time responsiveness, especially in high-speed networks, as processing outside the data plane incurs extra delays. P4 architecture constraints, such as a lack of floating-point support and limited memory, require that ML or hybrid systems also perform their detection outside the data plane, further reducing immediate response to potential threats.

Most current systems, whether P4-based or not, are restricted by a need for external processing and signature dependence [13], hindering their capacity to autonomously detect new or slow stealthy port scans. p4SD overcomes these gaps by introducing an efficient, hardware-optimized mechanism for detecting a broad range of port scans, including stealthy, low-frequency types, directly within the data plane, enabling timely threat response without relying on external components.

III. P4SD

p4SD is designed to detect port scanning activities by monitoring the evolution of probes over time based on the destination IP and transport layer port combinations of incoming traffic. Instead of relying on scan-specific signatures, it leverages a generic, pattern-based anomaly detection method built on fingerprint tracking and frequency analysis.

A. Architecture

The core architecture of p4SD, depicted in Figure 1, is inspired by the SWAMP algorithm [14], which uses a cyclic

fingerprint buffer with a frequency hash table in tandem to monitor temporal patterns continuously. Here, each incoming packet is processed by hashing its destination IP and port to generate a fingerprint (*i.e.*, using the CRC32 algorithm). This fingerprint is inserted into the cyclic buffer, replacing the oldest entry in a round-robin manner. Concurrently, the frequency table is updated to reflect the new fingerprint's addition and the outgoing one's removal. The system maintains a counter that estimates the number of unique fingerprints within the buffer cycle. This mechanism enables monitoring traffic patterns' evolution and provides the foundation for detecting deviations indicative of scanning activity.

Upon completion of a full buffer cycle, a detection function is invoked to identify potential anomalies. This function employs a weighted moving average to monitor fluctuations in the number of unique fingerprints between consecutive cycles. The objective is to identify significant positive deviations, which may signal port scanning behavior. Employing a moving average enables the system to track long-term behavioral trends, allowing it to detect slow scans that unfold over prolonged durations.

The detection logic, described in Algorithm 1, centers on the computation of an observation value, denoted as $obs(t)$, defined as the absolute difference between the current distinct count, $dist(t)$, and that of the previous cycle, $dist(t-1)$. This observation is used to update the variation metric, $var(t)$, where α is a tunable parameter within the range $[0,1]$ that determines the influence of recent observations relative to historical data.

An anomaly alarm is triggered when the increase in the distinct fingerprint count exceeds the previous variation scaled by a threshold parameter γ (see Lines 3-5). Note that the system is designed to raise alarms only when the distinct count increases, as port scans typically expand the range of targeted ports or hosts. This selective alarm mechanism reduces false positives and aligns with the typical expansion pattern of reconnaissance activities.

Algorithm 1 Anomaly detection using moving average

```

1:  $obs(t) \leftarrow |dist(t) - dist(t-1)|$ 
2:  $var(t) \leftarrow (1 - \alpha) \cdot var(t-1) + \alpha \cdot obs(t)$ 
3: if  $dist(t) - dist(t-1) > \gamma \cdot var(t-1)$  then
4:   if  $dist(t) > dist(t-1)$  then
5:     Raise ALARM

```

The proposed approach uses frequency windows based on the number of packets processed, rather than time-based windows, to determine when to trigger a change detection. This design simplifies memory management and ensures consistent operation across varying traffic rates.

p4SD uses hashed positions as identifiers to optimize memory efficiency rather than storing full fingerprints. Although this approach introduces the possibility of collisions, the trade-off is justified by the memory constraints in data planes and the need for real-time performance.

The parameters α (*i.e.*, the weight of new observations) and γ (*i.e.*, the threshold sensitivity) are configurable,

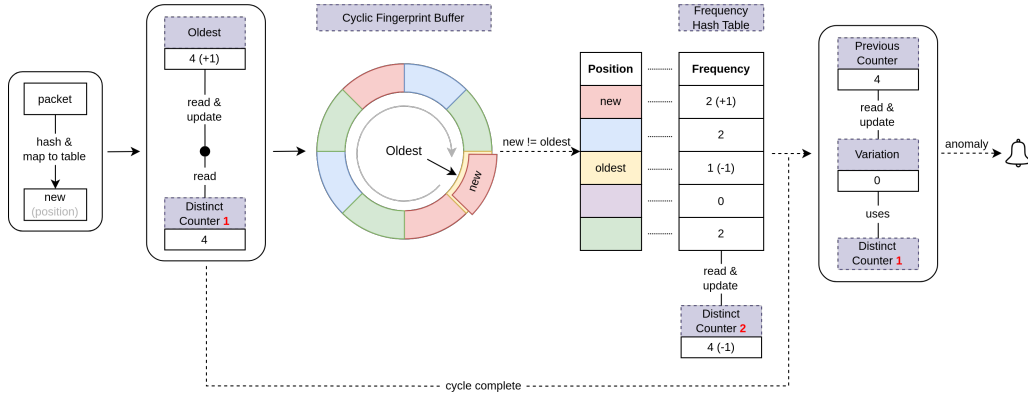


Fig. 1: p4SD detection architecture

allowing the system’s responsiveness to be tuned according to the operational environment. The buffer size, which determines the number of packets per cycle, can also be adjusted to match the expected traffic load. Section V discusses the impact of tuning these operational parameters.

B. Implementing p4SD with P4

p4SD leverages P4 towards detecting both slow and fast port scans by using resource-efficient strategies to overcome limitations such as the lack of floating-point arithmetic, loops, dynamic memory allocation, and limited pipeline depth and memory resources of hardware platforms. The proposed strategy replaces resource-demanding operations with integer-based arithmetic, static memory allocation, and deterministic packet-processing logic compatible with P4’s restricted execution environment.

Two proof-of-concepts were implemented: a software-based version using BMv2¹ and a hardware-based version on Netronome NFP-4000 SmartNICs. The BMv2 implementation provides single-threaded processing suitable for prototyping and virtualized environments. The Netronome implementation uses 480 parallel threads for line-rate processing, ideal for high-performance, large-scale packet processing. Both source codes are publicly available².

The core data structures in p4SD (§ III-A) are implemented as stateful registers, since P4 lacks dynamic memory allocation. The cyclic buffer and frequency table memory allocation balances scalability and collision risks. The cyclic buffer’s size can be adjusted according to network throughput, with each entry storing a hashed fingerprint. The frequency table’s entries track occurrence counts, which theoretically could match the cyclic buffer’s size in a worst-case scenario of all unique fingerprints in a cycle. However, in real-world deployments, this requirement is less stringent, as administrators only need to set the frequency table size to a value reasonably greater than the number of services offered by the target system.

Overly shrinking the frequency table raises collision risks, as multiple scan-related keys might map to the same entry, degrading detection accuracy. The lack of floating-point support is addressed by writing the weighted moving variation average equation in Algorithm 1 with bitwise shift operations. This replaces floating-point arithmetic with integer-based approximations, preserving detection accuracy while avoiding computational overhead. This strategy enables real-time scan detection by analyzing the variation of distinct entries between cycles without involving SDN controllers.

To prevent race conditions during concurrent register access in multi-threaded environments, p4SD implements fine-grained locking mechanisms for each entry in its data structures. This locking mechanism also enables p4SD compatibility with multiple hardware platforms by providing synchronization that abstracts hardware-specific characteristics, allowing seamless operation across diverse architectures. For single-element data structures, e.g., the oldest index, distinct counters, and moving variation, p4SD assigns a dedicated spin-lock to ensure that operations on one element do not block access to others. This ensures that the value used in the variation calculation accurately reflects the state up to that point, preventing it from being affected by incoming packets. While the counter could be read only at each reset, limitations in P4 made this approach more efficient in terms of throughput.

The partitioned design of more complex data structures, e.g., the cyclic buffer and the frequency table, allows for a finer-grained locking scheme. Specifically, each entry in these structures is assigned its own spin-lock, as illustrated in Figure 2(a) and 2(b). This approach enables operations on different entries in parallel, eliminating the need to lock the entire structure.

IV. TEST METHODOLOGY AND SCENARIOS

The evaluation of p4SD was conducted in a controlled environment that closely emulates real-world network conditions. This environment represents a small business

¹<https://github.com/p4lang/behavioral-model>

²<https://github.com/dissect-project/p4SD>

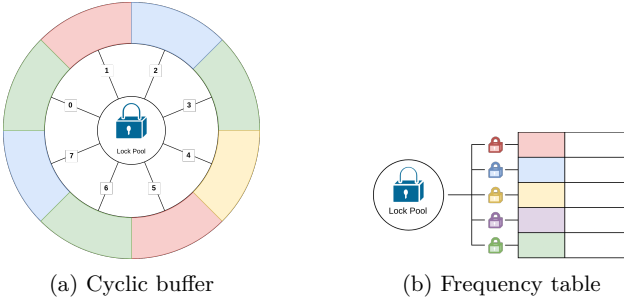


Fig. 2: Lock granularity in key system structures

network with four distinct services accessible via different TCP ports. The testbed was implemented using a Netronome NFP-4000 Agilio CX SmartNIC to assess performance under high-throughput conditions. Figure 3 illustrates the network scenario used to evaluate p4SD.

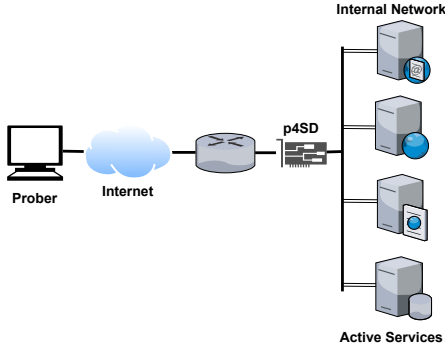


Fig. 3: Test environment - Network scenario

Traffic generation relied on two main tools: the *iPerf*³ generated regular network activity with variable packet rates and realistic service access patterns based on [15], and the *nmap*⁴ generated port scan traffic for both fast and slow scan behaviors by adjusting probe intervals. This combination enabled the creation of traffic patterns that included both legitimate usage and malicious scanning, reflecting an operational context.

For the slow port scan scenario, *nmap* was configured to scan the 1000 most common ports with probe delays of 10, 20, and 30 seconds [16], testing the system's ability to detect stealthy, low-rate scans. The fast port scan scenario involved scanning the same ports without delay, stressing the system's capacity to identify rapid bursts of anomalous activity. Another scenario introduced a sporadic service, which only received traffic every five minutes, to evaluate the system's resilience to false positives and its adaptability to irregular service access. Throughput and system's scalability were assessed by running p4SD under 1 Gbps

³<https://iperf.fr/>

⁴<https://nmap.org/>

and 10 Gbps traffic load, both with a single service and all four active services.

The evaluation methodology focused on four key performance metrics: detection accuracy, response time for identifying scanning activities, performance over heavy network load, and the influence of cyclic buffer size on the system's performance. Detection accuracy was assessed by comparing the number of alarms generated during port scans to the actual number of scan probes and by identifying false positives during continuous or sporadic service activity periods. Detection latency was measured as the time elapsed between the onset of anomalous activity and the triggering of an alarm. The performance was evaluated by measuring the traffic throughput while the p4SD processed packets toward single and multi-service scenarios.

Each test scenario began with a period of regular activity, followed by introducing scan traffic, and concluded with a return to regular traffic. This structure allowed for observation of the system's adaptation. All alarms generated by p4SD were logged with timestamps, enabling detailed post-test analysis. The system was configured with $\alpha = 0.7$ and $\gamma = 1$ for all scenarios described.

V. RESULTS

For the challenging slow port scan, Figure 4 presents the number of alerts for a constant throughput of 1 Gbps and cyclic buffer sizes between 400 and 10000. This scenario allows assessing the proposed base mechanism in detecting slow scans since it is ensured that at most one probe occurs per cycle. For all the analyzed probe frequencies, the number of expected alarms is up to 996, as the test scenario has 4 active legitimate services whose traffic to their ports should not be classified as a port scan.

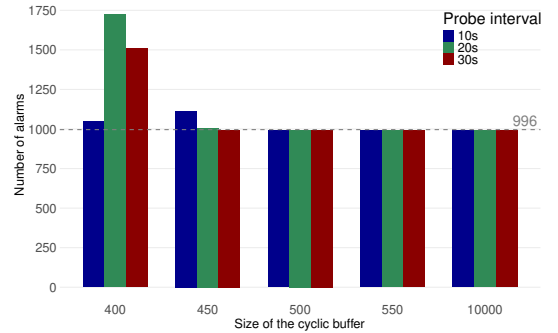


Fig. 4: Detection accuracy - Slow scan

As illustrated in Figure 4, the cyclic buffer with a size of 400 yields a higher rate of false positives, particularly for probe frequencies of 20s and 30s. This increase is closely tied to fluctuations in the number and composition of fingerprints generated by legitimate services in each monitoring cycle. Due to the high packet volumes processed by active services within short time windows, slight

variations in packet arrival order can cause certain service fingerprints to be omitted in one round and reappear in the next. For example, one cycle may contain fingerprints for three services, while the subsequent cycle presents four, despite all being associated with legitimate traffic. The system, interpreting this discrepancy as a potential anomaly, raises an alarm. However, since this variation is expected behavior for active services under normal network conditions, these alarms are ultimately classified as false positives.

p4SD successfully detects over 99% of the anomalies related to the 996 ports scanned in slow scan scenarios. Here, such accuracy is achieved with a cyclic buffer of size 500 and remains stable up to a cyclic buffer of size 10000, regardless of the scan frequency. While the performance could be further enhanced by increasing the frequency table size, such an increase is unnecessary, given that the current number of detected anomalies is likely more than enough for an SDN controller to identify the behavior as a port scan.

The sporadic service scenario poses additional challenges for slow-scan detection, since it may be incorrectly classified as a probe due to its irregular appearance across sequential cycles rather than maintaining a consistent presence. As presented in Figure 5, this scenario indeed leads to a higher number of alarms, around 25% for lower cyclic buffer sizes and 10% for higher cyclic buffer sizes. Such a difference is justified by the extended time to complete bigger buffers, which will eventually include the sporadic service fingerprint.

To reduce false positives, two main strategies can be used: configuring the data plane to ignore alarms from known services, though limited by static, compile-time service lists, or using correlation algorithms in the SDN controller to match alarm patterns with service fingerprints over multiple monitoring cycles. The latter assumes isolated alarms are insufficient and relies on fingerprint correlation to distinguish scans from legitimate sporadic activity. This method retains data plane efficiency, uses the controller's broader network view for smarter detection, and balances accuracy with real-time performance.

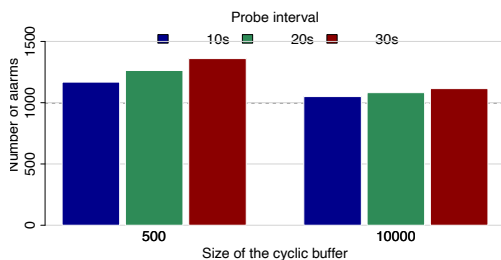


Fig. 5: Detection accuracy with sporadic services

As expected, fast port scan detection shows different

characteristics compared to slow scans. As shown in Figure 6, the number of alarms generated for variable cyclic buffer sizes is significantly lower than that of the slow port scan tests. This is because in a fast port scan, different ports are scanned in a very short period, so its occurrence takes place over fewer cycles than in a slow scan. Similarly to the slow scan detection, the number of alarms stabilizes at a cyclic buffer size of 500.

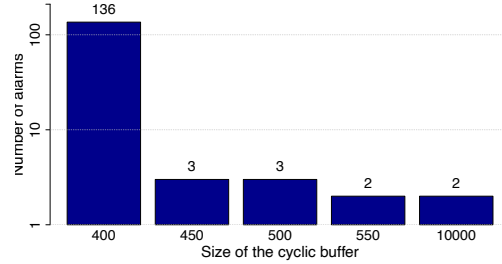


Fig. 6: Alarms for fast scan

Regarding detection latency, the performance of p4SD is fundamentally tied to the cyclic buffer size and network traffic patterns, which determine how many packets are processed before an anomaly check is performed. As demonstrated in Figure 7, at 1 Gbps throughput with an average of 2936 packets per second, the approximate time to complete a full cycle varies significantly based on buffer configuration. For the tested cyclic buffer sizes, the detection latency ranges from 0.13 seconds for a buffer size of 400 fingerprints to 3.40 seconds for 10,000 fingerprints. These values represent the maximum detection latency, as anomalous packets such as port scan probes may arrive at any point during a cycle.

Importantly, all tested cyclic buffer sizes provide detection latency shorter than the delays typically used in slow port scan scenarios. Ensuring that the proposed system can identify individual probes within their temporal context while maintaining the ability to correlate them over time through the moving average mechanism.

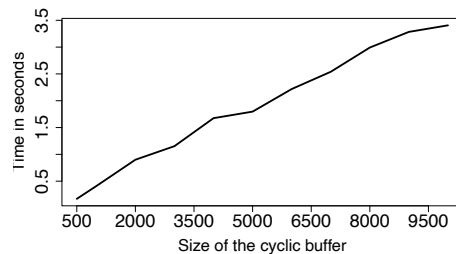


Fig. 7: Detection delay per buffer size

The trade-off between detection latency and system stability becomes evident when comparing buffer sizes for

different scan strategies. Smaller buffers provide faster detection but increase the likelihood of missing legitimate services, leading to false positives. Conversely, larger buffers ensure more comprehensive service coverage but introduce slightly higher detection delays. For fast scans, since the time between cycles is shorter for smaller buffer sizes, p4SD can generate alerts faster. It occurs due to the higher and sudden variation of the number of different fingerprints between two consecutive cycles. These results highlight the importance of fine-tuning buffer sizes according to the link capacity.

The analysis of the number of fingerprint collisions reveals that for a frequency table of size 400, the average of all the test scenarios is 0.003 collisions per cycle, and for a table of size 10,000, the average is 0.0002 collisions per cycle. Although these numbers can be reduced by increasing the frequency table size, this increase is unnecessary since the current detected anomalies sufficiently identify different port scan behaviors.

Performance evaluation demonstrates the system's practical viability without compromising network throughput. As demonstrated in Figure 8, tests with the Netronome SmartNIC, using both single and multiple services (four services), show that the NIC sustains traffic throughput close to its nominal capacity (10 Gbps). This confirms the solution's effectiveness in detecting multiple port scan types without impacting Quality of Service (QoS), while maintaining scalability. Detection occurs practically in real-time for all cyclic buffer sizes, allowing prompt action against imminent threats.

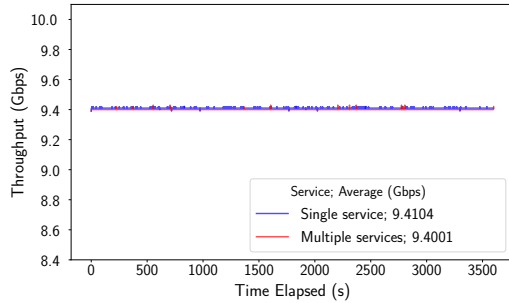


Fig. 8: Performance in hardware deployment

VI. CONCLUSION

This paper introduces p4SD, a lightweight port scan detection system for programmable SDN data planes. Using P4's programmability with cyclic fingerprint tracking and frequency analysis, p4SD detects both fast and slow port scans directly in the data plane with over 99% accuracy. The system employs integer-based arithmetic, stateful registers, and fine-grained locking to overcome P4 limitations while maintaining hardware compatibility, effectively identifying even slow scan attacks designed to evade traditional detection methods.

Evaluation shows p4SD's practical viability with near-hardware throughput, real-time detection, and minimal latency. It offers adjustable buffers for network adaptability and advances programmable network security without performance loss. While porting to different hardware would require adapting to target-specific pipeline architectures, memory models, and concurrency mechanisms, the fundamental detection logic remains viable in any P4-compatible device. Future work will target reducing false positives via SDN controller correlation algorithms.

Acknowledgment: This work is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., under the support UID/50014/2023 (<https://doi.org/10.54499/UID/50014/2023>).

REFERENCES

- [1] S. Panjwani, S. Tan, K. Jarrin, and M. Cukier, "An experimental evaluation to determine if port scans are precursors to an attack," in *2005 International Conference on Dependable Systems and Networks (DSN'05)*, Jun. 2005.
- [2] E. Bou-Harb, M. Debbabi, and C. Assi, "Cyber scanning: a comprehensive survey," *Ieee communications surveys & tutorials*, vol. 16, no. 3, 2013.
- [3] H. Wu, Z. Shao, G. Cheng, X. Hu, J. Ren, and W. Wang, "Detecting slow port scans of long duration in high-speed networks," in *2022 IEEE Global Communications Conference*, 2022.
- [4] B. Lewis, M. Broadbent, and N. Race, "P4id: P4 enhanced intrusion detection," in *2019 IEEE conference on network function virtualization and software defined networks (NFV-SDN)*. IEEE, 2019.
- [5] K. Tavares and T. C. Ferreto, "P4-onids: A p4-based nids optimized for constrained programmable data planes in sdn," *Anais do XXXIX Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, 2021.
- [6] A. S. Ilha, A. C. Lapolli, J. A. Marques, and L. P. Gaspary, "Euclid: A Fully In-Network, P4-Based Approach for Real-Time DDoS Attack Detection and Mitigation," *IEEE Transactions on Network and Service Management*, vol. 18, no. 3, sep 2021.
- [7] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, 2005.
- [8] Y. Chabchoub, R. Chiky, and B. Dogan, "How can sliding hyperloglog and ewma detect port scan attacks in ip traffic?" *EURASIP Journal on Information Security*, 2014.
- [9] B. Groza and P.-S. Murvay, "Efficient intrusion detection with bloom filtering in controller area networks," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 4, 2018.
- [10] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than bloom," in *10th ACM International Conference on Emerging Networking Experiments and Technologies*, 2014.
- [11] J.-S. Lee, Y.-Y. Fan, C.-H. Cheng, C.-J. Chew, and C.-W. Kuo, "ML-based intrusion detection system for precise apt cyber-clustering," *Computers & Security*, vol. 149, 2025.
- [12] M. A. Talukder, M. M. Islam, M. A. Uddin, K. F. Hasan, S. Sharmin, S. A. Alyami, and M. A. Moni, "Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction," *Journal of big data*, vol. 11, 2024.
- [13] F. Musumeci, A. C. Fidanci, F. Paolucci, F. Cugini, and M. Tornatore, "Machine-learning-enabled ddos attacks detection in p4 programmable networks," *Journal of Network and Systems Management*, vol. 30, 2022.
- [14] E. Assaf, R. B. Basat, G. Einziger, and R. Friedman, "Pay for a Sliding Bloom Filter and Get Counting, Distinct Elements, and Entropy for Free," Apr. 2018.
- [15] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, and A. Hotho, "Creation of flow-based data sets for intrusion detection," *Journal of Information Warfare*, vol. 16, no. 4, 2017.
- [16] M. u. Nisa and K. Kifayat, "Detection of slow port scanning attacks," in *2020 International Conference on Cyber Warfare and Security (ICWS)*, 2020.